S.M.A.R.T. Drive Triage

An evidence-based generative AI application for interpreting storage-drive reports

Introduction to Generative AI | Individual project

Lis Hadërgjonaj

Neptun code: B5KCCW

University of Debrecen

Faculty of Informatics

Computer Science Engineering BSc

Project summary

I developed S.M.A.R.T. Drive Triage to turn technical storage-drive telemetry into a readable, evidence-based assessment. My application accepts a pasted report, extracts the supplied measurements, assigns a triage category, and explains the findings through a Streamlit interface. I use a Groq-hosted language model for interpretation and Python for response validation, presentation, and predefined follow-up guidance.

The central design decision is to separate interpretation from action. I allow the model to interpret the report, but I do not allow it to freely write the displayed recommendations. That distinction grew directly out of testing: some early outputs sounded convincing while making claims the supplied evidence could not support.

This report documents my completed course-project version. The accompanying testing appendix preserves the development runs, output excerpts, and available captures.

1. Problem and purpose

A S.M.A.R.T. report is useful only if its measurements are interpreted carefully. The text can mix normalized values, thresholds, raw counters, hexadecimal flags, and manufacturer-specific attributes. A reader can easily mistake a large raw value for a large number of failures, treat PASSED as an overall reliability guarantee, or interpret a single counter as proof that a drive is getting worse.

I chose this problem because it requires more than a generic question-and-answer response. My application must identify the report format, extract evidence, distinguish different kinds of values, choose a category, and explain uncertainty without inventing measurements or turning a snapshot into a prediction.

My objective was not to replace a hardware specialist. I wanted a clear first-pass interpretation that stays close to the supplied report and helps a non-expert understand what deserves attention.

2. Application workflow

Stage	What I implemented
Input	A multiline Streamlit text box accepts report text. I reject empty input before making an API request.
Generation	One Groq request combines my system prompt with the report. I request a JSON object using the configured model and generation settings.
Validation	Python parses the JSON and checks the assessment label, field types, and evidence-row structure.
Presentation	The interface displays a category-specific summary, an evidence table, recommended actions, and Important Notes.
Action control	Python selects recommendations from a predefined dictionary using the returned category. The model does not write this displayed section.

The application processes text only. It does not scan drives, run commands, change firmware or hardware settings, keep a history database, or provide a built-in export function. The report captures in my testing appendix are documentation evidence, not an export feature of the application.

3. Technology and architecture

Component	Selection and role
Language	Python. I used Python 3.13 during development; this is not an exclusive version requirement.
Interface	Streamlit 1.66.1 provides the text input, parameter controls, status messages, and result display.
API library	Groq Python library 1.7.0 handles requests to the hosted model.
Model	openai/gpt-oss-20b, configured through the Groq API.
Data exchange	JSON Object Mode, followed by application-side parsing and structural validation.

Component	Selection and role
Repository	GitHub stores the application and project materials. I keep credentials and local environments out of the repository.

I kept the architecture deliberately small: one application file, pinned direct dependencies, a private configuration file for the API key, and supporting documentation. I did not need a separate backend service or a local model installation for this workflow.

The request contains a system message with my interpretation rules and a user message containing the report. I explicitly treat report text as untrusted data rather than instructions. This is an instruction-level boundary; the final six-case set did not establish robustness against adversarial reports.

4. Prompt engineering

I used role assignment and few-shot prompting as the two principal techniques. I supported them with an explicit response contract and evidence-handling constraints. I revised the prompt in response to observed failures rather than relying on the initial wording.

4.1 Role assignment

I assigned the model the role of a cautious storage reliability engineer. The role establishes the intended perspective, but testing showed that a role alone does not prevent overconfident language. I therefore translated caution into specific rules about measurements, trends, timing, and vendor encoding.

4.2 Few-shot examples

I included examples for missing telemetry, concerning SATA sector counts, an unexplained raw read-error value, no-warning NVMe fields, a warning NVMe report, and unrelated input. Each example pairs an input with the intended structured response. I also state that these examples are illustrative, so their measurements are not evidence about the submitted report.

The examples teach distinctions that are easy to miss: pending and uncorrectable counts may overlap; a raw counter may require vendor documentation; and NVMe Percentage Used describes estimated endurance consumption rather than occupied storage capacity.

4.3 Evidence constraints

I require the model to preserve reported values as strings, distinguish raw and normalized columns, avoid replacing missing measurements with zero, and avoid guessing undocumented encodings. I prohibit unsupported claims of deterioration, imminent failure, remaining years, and broad overall normality. I also prohibit recommendations, commands, device paths, and diagnostic instructions anywhere in the returned JSON.

For recurring counter descriptions, I give specific neutral wording. Power-cycle and operating-hour totals should not become judgments about acceptable or excessive usage. I separately name warning-temperature and critical-temperature counters to prevent label mixing.

4.4 Explicit category guidance

My categories are No obvious warning in supplied data, Monitor, Urgent attention, Insufficient information, and Invalid input. These are application triage labels, not failure predictions.

One early NVMe result recognized below-threshold spare capacity but selected Monitor. I added an explicit rule for the tested combination of FAILED overall health and spare below its supplied threshold: select Urgent attention. This is an explicit sufficient-condition instruction for that case, not a complete independent diagnostic engine or an exhaustive definition of every category.

5. Structured output and controlled presentation

I chose JSON because I wanted the application, rather than the model, to control the page layout. The response contains assessment, summary, evidence, and limitations. Each evidence row contains attribute, reported_value, and interpretation. The internal limitations key is displayed under the friendlier heading Important Notes.

Field	Use in the application
assessment	Validated against the five allowed labels and used to choose the fixed summary and actions.
summary	Requested and validated as a string. The current interface uses a predefined summary instead of displaying this generated field.
evidence	Displayed as a table after row-structure and string-type checks.
limitations	Validated as a list of strings and displayed as Important Notes.

My validator checks whether the response can be used by the interface. It does not independently prove that a reported value was copied correctly or that an interpretation is justified. JSON Object Mode is not the same as factual validation, and my current function does not enforce an exact-key schema or reject every unsupported statement.

I use predefined recommendations because early free-text outputs reintroduced diagnostic commands and overstated what a test could establish. For concerning categories, my fixed actions prioritize protecting data before device-appropriate follow-up. This controls the recommendation wording, although the model still determines which category selects those actions.

6. Parameters and response handling

I expose temperature from 0.0 to 1.0 and the maximum completion-token setting from 1024 to 4096. The defaults are 0.2 and 2048, respectively. My recorded final regression set used those defaults. These controls parameterize generation at request time; they are not model training or fine-tuning.

I did not record a controlled experiment comparing multiple parameter values, so I do not claim that the defaults are an experimentally optimal configuration. My experimentation focused on prompt wording, examples, interpretation rules, and repeated outputs.

I check the completion finish reason before displaying an assessment. When the response reaches its length limit, the application asks for a larger output budget instead of presenting a partial result. I also implemented handling for empty input, missing or empty credentials, authentication failures, connection failures, rate limits, API failures, invalid structured responses, and unrecognized labels. I distinguish implemented handlers from behavior actually verified in the retained tests.

7. Development findings and revisions

The most important development lesson was that a plausible explanation can still exceed the evidence. I did not accept an output merely because it chose an appropriate category or included a

disclaimer at the end.

Observed issue	What I changed
Trend and timing overclaims	I added explicit snapshot constraints and used fixed summaries that do not assert deterioration or imminent failure.
Commands and diagnostic promises	I moved displayed recommendations into fixed Python text and prohibited diagnostic instructions in generated fields.
Capacity/endurance confusion	I added NVMe-specific wording and examples distinguishing endurance consumed from storage occupancy.
Warning category too weak	I added the explicit failed-health and below-threshold-spare priority rule and retested with changed values.
Unsupported usage judgments	I replaced general neutrality guidance with specific descriptions for cycles and hours.
Mixed temperature counters	I provided distinct interpretation wording for warning and critical temperature-time counters.
Irrelevant zero-error notes	I required notes to match the supplied evidence and removed a reusable example sentence about error timing.

I kept repeated runs separate. In one pair of unchanged-input tests, the category and endurance interpretation were correct while the counter descriptions still said there was no indication of abnormal usage. That observation mattered: describing a counter as neutral does not remove an unsupported judgment attached to it.

I also changed numeric values in later tests. The warning NVMe retests used different spare percentages and error counts so that I could check value preservation and rule application rather than literal reproduction of a worked example.

8. Testing and Results

I used synthetic inputs so that I could control the expected evidence and avoid including real drive identifiers. I retained both successful and unsuccessful development outputs. The detailed appendix records 20 identifiable runs, including early revisions, repeated inputs, and the final six cases. It also separates historical observations whose original-run mapping has not been recovered; I have not invented evidence to fill those gaps.

For the final regression set, I kept the consolidated prompt unchanged and evaluated the complete displayed output against case-specific checks. Cropped screenshots do not independently show every setting, so I distinguish the recorded test configuration from what each capture visibly proves.

Case	Observed category	Result checked
Concerning SATA counters	Urgent attention	Separate pending/uncorrectable counts; no unsupported trend or timing; data protection first.
Unknown raw encoding	Insufficient information	Raw value retained without a literal failed-read count, wear diagnosis, or unnecessary diagnostic advice.

Case	Observed category	Result checked
No-warning NVMe	No obvious warning in supplied data	Endurance interpretation, neutral counters, separate temperature labels, and relevant zero-error notes.
Warning NVMe	Urgent attention	6% spare below 10%; 82% endurance consumed; two counts of 9 kept separate; no failure prediction.
Missing telemetry	Insufficient information	No invented measurements or fault; no commands or replacement advice.
Unrelated request	Invalid input	No poem or invented telemetry; request for an appropriate report.

All six final cases met their predefined checks in the retained outputs. This is a targeted regression result, not a general diagnostic-accuracy score. Some cases resemble the few-shot examples, and each final case was observed once in this consolidated set. Earlier repetitions belong to preceding prompt revisions. The six cases do not independently establish Monitor-category coverage, comprehensive manufacturer coverage, or adversarial-input robustness.

I include the full experiment record separately so that the main report remains readable. The appendix provides input definitions, output excerpts, individual outcomes, revisions, and available original captures.

9. Installation and use

I pin the direct dependencies in requirements.txt as streamlit==1.66.1 and groq==1.7.0. My development environment used Python 3.13; another version can be used when compatible with those packages and their dependencies. I do not claim that I tested every compatible Python version.

On Windows, the setup sequence is to obtain the project files, select a compatible Python installation, create a virtual environment, and install requirements.txt. I then configure GROQ_API_KEY in .streamlit/secrets.toml and launch Streamlit. The README provides the complete copyable commands and configuration example.

For normal use, I remove identifying details from the report, enter the text, keep or adjust the generation settings, and select Analyze. The text box may show Streamlit’s Ctrl+Enter hint; applying the text is separate from selecting the Analyze button. The interface is designed to start analysis through that button.

I read the category together with its evidence and notes. Urgent attention means that the findings warrant prompt data protection and follow-up; it does not specify a failure time. No obvious warning in supplied data is a bounded statement about the supplied measurements, not an overall reliability guarantee.

10. Data handling and assessment scope

The report text is sent to Groq. I display a notice asking for serial numbers and other identifying information to be removed before submission. I keep the API key in a private secrets file and exclude that file and the local virtual environment from the repository. Repository exclusions do not replace checking files selected for manual uploads.

I do not implement a report-history database in the application. That describes my application, not a claim about the external provider’s data-retention policies. I make no undocumented promise that

submitted text is never retained by the provider.

I designed the assessment around the supplied snapshot. Historical change, exact failure timing, undocumented raw-value meanings, and comprehensive hardware diagnosis are outside the evidence available to this workflow. Predefined guidance improves consistency, but it does not independently verify the model's selected category. These boundaries are part of the submitted design, not a commitment to future development.

11. Conclusion

I completed a standalone generative AI application with an interactive frontend, a hosted-model API call, adjustable generation settings, structured response validation, and controlled presentation. The model performs substantive extraction and categorization rather than simply answering an unrestricted question.

The development work was as important as the final interface. I identified where the output overstated the evidence, revised the prompt and application structure, and checked the results through changed-value and repeated-input tests. My final regression set demonstrates the intended behavior for the six documented cases, while the appendix preserves how I reached that version.

The submitted materials consist of the application, pinned dependencies, README, this project report, and the detailed testing appendix. Together they document what I built, the decisions I made, and the tests I performed.